ee2004

Week3,4

testbench

wa2.s * | wa2_tb.s *

```
1   ##TESTBENCH
2   #TEST 1
3   IN R0 is 1
4   IN R1 is 2
5   OUT R2 is 3
6   >>;          Test 1 PASSED.
7   #TEST 2
8   IN R0 is 10
9   IN R1 is 200
10  OUT R2 is 210
11  >>;          Test 2 PASSED.
```

wa2.s * | wa2_tb.s *

```
1
2       ADD     R2,R1,R0
3
4
```



A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        B       START
```

```
int* p = (int*)0x1000;
while(1){
    *p++;
}
```

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        CMP     R0, #100
        BNE     START
```

```
int* p = (int*)0x1000;
while(*p != 100){
    *p++;
}
```

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

```
1   main
2           mov     R1,#0 ; R1 is i
3           mov     R0,#0 ; R0 is sum
4
5   loop
6           add     R0,R0,R1 ; sum = sum + i
7           add     R1,R1,#1 ; i++
8           cmp     R1,#10 ; i <= 10
9           ble     loop
10
11  halt
12          b       halt
```

```c
int x[10];
int sum = 0;

for (int i = 0; i < 10; i++)
    sum += x[i];
```

```
1   veri    DCD     1, 3, 5, 7, 9, 11, 13, 15, 17, 19
2
3   sum     DCD     0
4
5   main    mov     r0,#veri
6           mov     r1,#sum
7           ldr     r2,[r1]
8           mov     r3,#0 ; r3 is i
9
10  for     ldr     r4,[r0,r3,lsl #2]
11          add     r2,r2,r4
12          add     r3,r3,#1
13          cmp     r3,#10
14          blt     for
15          str     r2,[r1]
16
17  halt    b       halt
```

```
        MOV     R3, #5
        MOV     R4, #5


        CMP     R3,R4     ; if (i >= j)
        BLT     else
        SUB     R0,R3,R4  ; x = i - j;
        B       endif     ; else
else    SUB     R0,R4,R3  ; x = j - i;
endif

        CMP     R3,R4     ; x = (i >= j) ? i - j : j - i;
        SUBGE   R0,R3,R4
        SUBLT   R0,R4,R3
```

```
2   main
3           mov     r0, #35
4           mov     r1, #55
5           adr     r5, 0x300
6           bl      GCD
7
8   GCD
9           cmp     r0,r1
10          beq     halt
11          subgt   r0,r0,r1
12          sublt   r1,r1,r0
13          b       GCD
14
15  halt
16          str     r0, [r5]
17
```

```
int gcd(a,b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}

int x = 35;
int y = 55;
int z;

z = gcd(x, y);
```

Creating an Assembly Project in STM32CubeIDE

- We provide a template assembly code using the mentioned directives in Listing 15.1. To note here, this structure is specific to STM32CubeIDE. There may be extra directives or a different format for other development environments. Therefore, the reader should check them before preparing his or her assembly code in other IDEs.

- **.text** assembles the following code into text (executable code) section.
- **.align** defines the length of instructions to be processed in terms of bytes.
- **.global** defines one or more global symbols.
- **.end** identifies end of the code.

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

- The first directive .text indicates that the following lines correspond to assembly code. Hence, they should be placed in the code section of the flash memory.
- The second directive .align defines the length of instructions to be processed in terms of bytes. If the instructions to be executed have 16-bit length, then we should use .align 2 as in our case.
- The third directive .global defines the global variable main which indicates the starting address of our code. Finally, the directive .end indicates the end of the assembly code.

Creating an Assembly Project in STM32CubeIDE

- There is no direct method to create an assembly project in STM32CubeIDE.
- Therefore, we should first follow the steps to form an empty C project.
- To create the debug file (with extension ".debug"), we should debug our (empty) C code for the first time.
- As the mentioned file is created, we should delete the "main.c" file from the "Src" folder of the project.
- In fact, we can delete all files with extension ".c" from the project's "Src" folder.
- Then, we should add a new file with extension ".s" to this folder.
- To do so, right-click on the "Src" folder, and press "New" and "File."
- In the opening window, enter the "File Name" such as "main.s" and click "Finish."
- An empty file will be created under the "Src" folder.
- We can add our assembly code to this file. Here, we will use the sample codes:

- As we compile the code, we can execute it either by the “step into” option in STM32CubeIDE or directly by the “Resume” command. We can observe output of the code; the registers have been modified according to our assembly code.
- We can apply operations on the memory of the STM32F4 microcontroller by assembly language. In the second code, we modify and reach content of the memory address 0x20000020 by the LDR and STR instructions. As we compile and run the code, we can observe its output in the “Registers” and “Memory Browser” Windows.

```
.text
.align 2
.global main

Mem_Addr=0x20000020

main:

 MOVS R0, #0x09
 MOVS R1, #0x11
 MOVS R2, #10

 LDR R3, =Mem_Addr
 STR R2, [R3]
 LDR R0, [R3]

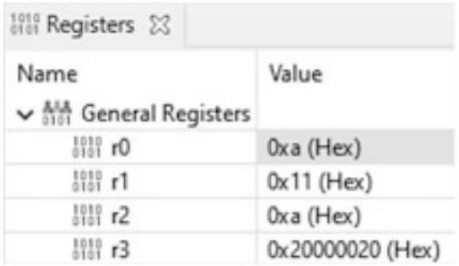
END: B END
  .end
```

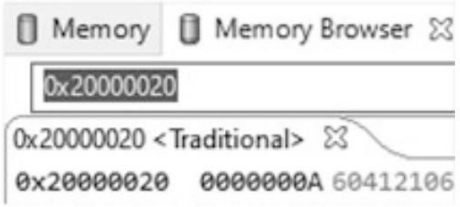
```
.text
.align 2
.global main

main:

 MOVS R0, #0x12
 MOVS R1, #0x2
 MOVS R2, #0x4
 ADD R3, R0, R1

END: B END
  .end
```





Lab1 solution

```
1
2
3           MOV     R0, #TESTDATA
4           MOV     R3, #8
5
6   loop    LDRB    R1, [R0]
7           ADD     R0, R0, #1 ; adres
8           ADD     R2, R2, R1 ; sonuç
9           SUB     R3, R3, #1 ; indeks
10          CMP     R3, #0
11          BNE     loop
12
13          LSR     R4, R2, #3
14
15          ADR     R5, 0x300
16          STR     R4, [R5]
17
18
19  TESTDATA DCD    0x0F1F2F3F, 0x4F5F6F7F
```